249057766 Week-1

MA7022 Data Mining and Neural Networks 2025-26 SEM2

In [ ]:

Week 1 — Data Wrangling and Exploratory Data Analysis

DMNN Module

Dataset: Online Retail II

This notebook explores the dataset structure, identifies data quality issues, and performs initial exploratory analysis before modelling.

In [1]:
```
import pandas as pd

# load dataset
data = pd.read_csv("online_retail_II (1).csv", encoding="ISO-8859-1")

# show first rows
data.head()
```

Out[1]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Countı
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	01/12/2009 07:45	6.95	13085.0	Unite Kingdo
1	489434	79323P	PINK CHERRY LIGHTS	12	01/12/2009 07:45	6.75	13085.0	Unite Kingdo
2	489434	79323W	WHITE CHERRY LIGHTS	12	01/12/2009 07:45	6.75	13085.0	Unite Kingdo
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	01/12/2009 07:45	2.10	13085.0	Unite Kingdo
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	01/12/2009 07:45	1.25	13085.0	Unite Kingdo

In [2]:
```
data.shape
```

Out[2]:
```
(525461, 8)
```

In [3]:
```
data.columns
```

Out[3]:
```
Index(['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'Price', 'Customer ID', 'Country'],
      dtype='object')
```

In [4]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   Invoice      525461 non-null  object
 1   StockCode    525461 non-null  object
 2   Description  522533 non-null  object
 3   Quantity     525461 non-null  int64
 4   InvoiceDate  525461 non-null  object
 5   Price        525461 non-null  float64
 6   Customer ID  417534 non-null  float64
 7   Country      525461 non-null  object
dtypes: float64(2), int64(1), object(5)
memory usage: 32.1+ MB
```

Task 2 — Unit of Analysis

The Online Retail dataset is transactional, meaning each row represents a line item within a purchase invoice. However, the data can be analysed at different levels depending on the research objective. Below are several possible units of analysis and their implications.

1. Invoice Line (Row Level)

Information gained: At this level, each row represents a specific product purchased in a transaction. It provides detailed information about product descriptions, quantities purchased, prices, and transaction timestamps.

Information lost: This level does not capture the overall context of the purchase basket. It also does not represent the behaviour of customers across multiple purchases.

2. Invoice / Basket Level

Information gained: By grouping rows using the Invoice number, we can analyse baskets of products purchased together. This allows us to examine basket size, total invoice value, and combinations of products bought in the same purchase.

Information lost: When data is aggregated at the invoice level, detailed information about individual product line items may be reduced or lost.

3. Customer Level

Information gained: Aggregating data by Customer ID allows us to analyse customer purchasing behaviour over time. This includes metrics such as total spending, purchase frequency, and customer lifetime value.

Information lost: At this level, detailed information about specific invoices or individual product purchases may be lost when transactions are aggregated.

Final Decision: Chosen Unit of Analysis

For the subsequent analysis in this project, the customer level will be used as the primary unit of analysis. This choice allows us to better understand purchasing behaviour over time and identify patterns such as repeat purchases, customer value, and purchasing frequency.

Customer-level analysis is also more useful for future modelling tasks such as customer segmentation or predicting future purchasing behaviour.

In [5]:
```
data.isnull().sum()
```

Out[5]:
```
Invoice             0
StockCode           0
Description      2928
Quantity            0
InvoiceDate         0
Price               0
Customer ID    107927
Country             0
dtype: int64
```

Task 3 — Data quality audit

Issue 1 — Missing Customer ID

Some transactions do not contain a Customer ID. This means we cannot link these purchases to a specific customer.

Why this matters: Customer-level analysis requires knowing which customer made each purchase. Missing IDs prevent tracking customer behaviour over time.

Planned handling: Rows with missing Customer ID will be removed when performing customer-level analysis, because they cannot be assigned to a specific customer.

In [6]:
```
data[data['Quantity'] < 0].head()
```

Out[6]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Cour
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	01/12/2009 10:33	2.95	16321.0	Austr
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	01/12/2009 10:33	1.65	16321.0	Austr
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	01/12/2009 10:33	4.25	16321.0	Austr
181	C489449	21896	POTTING SHED TWINE	-6	01/12/2009 10:33	2.10	16321.0	Austr
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	01/12/2009 10:33	2.95	16321.0	Austr

In [7]:
```
(data['Quantity'] < 0).sum()
```

Out[7]:
```
np.int64(12326)
```

Issue 2 — Negative Quantities (Returns)

Some rows have negative quantity values. These represent returned items rather than purchases.

Why this matters: If returns are treated as normal purchases, they will distort calculations such as total sales, basket size, and revenue.

Planned handling: Returns will be separated from purchase transactions. For most purchasing behaviour analysis, only positive quantities will be used.

In [8]:
```
data[data['Invoice'].astype(str).str.startswith('C')].head()
```

Out[8]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Cour
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	01/12/2009 10:33	2.95	16321.0	Austr
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	01/12/2009 10:33	1.65	16321.0	Austr
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	01/12/2009 10:33	4.25	16321.0	Austr
181	C489449	21896	POTTING SHED TWINE	-6	01/12/2009 10:33	2.10	16321.0	Austr
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	01/12/2009 10:33	2.95	16321.0	Austr

In [9]:
```
data['Invoice'].astype(str).str.startswith('C').sum()
```

Out[9]:
```
np.int64(10206)
```

Issue 3 — Cancelled Invoices

Some invoice numbers begin with the letter "C", which indicates that the transaction was cancelled.

Why this matters: Cancelled invoices represent reversed transactions and should not be treated as normal purchases.

Planned handling: Cancelled invoices will be excluded from purchase behaviour analysis to avoid incorrect revenue or quantity calculations.

In [10]:
```
data.describe()
```

Out[10]:

	Quantity	Price	Customer ID
count	525461.000000	525461.000000	417534.000000
mean	10.337667	4.688834	15360.645478
std	107.424110	146.126914	1680.811316
min	-9600.000000	-53594.360000	12346.000000
25%	1.000000	1.250000	13983.000000
50%	3.000000	2.100000	15311.000000
75%	10.000000	4.210000	16799.000000
max	19152.000000	25111.090000	18287.000000

Issue 4 — Extreme Values

Some transactions contain unusually large quantities or prices.

Why this matters: Extreme values may represent bulk purchases, errors, or unusual transactions that can affect averages and statistical summaries.

Planned handling: Extreme values will be examined carefully during analysis rather than removed automatically.

Task 4 — Minimal Data Cleaning

In [11]:
```
clean_data = data.copy()
```

Remove rows with missing Customer ID

In [12]:
```
clean_data['Customer ID'].isnull().sum()
```

Out[12]:
```
np.int64(107927)
```

In [13]:
```
clean_data = clean_data.dropna(subset=['Customer ID'])
```

In [14]:
```
clean_data['Customer ID'].isnull().sum()
```

Out[14]:
```
np.int64(0)
```

Remove cancelled invoices

In [15]:
```
clean_data['Invoice'].astype(str).str.startswith('C').sum()
```

Out[15]:
```
np.int64(9839)
```

In [16]:
```
clean_data = clean_data[~clean_data['Invoice'].astype(str).str.startswith('C
```

In [17]:
```
clean_data['Invoice'].astype(str).str.startswith('C').sum()
```

Out[17]:
```
np.int64(0)
```

Remove negative quantities (returns)

In [18]:
```
(clean_data['Quantity'] < 0).sum()
```

Out[18]:
```
np.int64(0)
```

In [19]:
```
clean_data = clean_data[clean_data['Quantity'] > 0]
```

In [20]:
```
(clean_data['Quantity'] < 0).sum()
```

Out[20]:
```
np.int64(0)
```

Convert InvoiceDate to datetime

In [21]:
```
clean_data['InvoiceDate'] = pd.to_datetime(clean_data['InvoiceDate'], dayfir
```

In [22]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 407695 entries, 0 to 525460
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   Invoice      407695 non-null  object
 1   StockCode    407695 non-null  object
 2   Description  407695 non-null  object
 3   Quantity     407695 non-null  int64
 4   InvoiceDate  407695 non-null  datetime64[ns]
 5   Price        407695 non-null  float64
 6   Customer ID  407695 non-null  float64
 7   Country      407695 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 28.0+ MB
```

Convert InvoiceDate to Datetime

The InvoiceDate column was originally stored as a string.
It was converted to datetime format so that time-based analysis such as monthly trends and purchase frequency can be performed correctly.

The parameter `dayfirst=True` was used because the dataset stores dates in day/month/year format.

In [23]:
```
clean_data['InvoiceDate'].head()
```

Out[23]:
```
0   2009-12-01 07:45:00
1   2009-12-01 07:45:00
2   2009-12-01 07:45:00
3   2009-12-01 07:45:00
4   2009-12-01 07:45:00
Name: InvoiceDate, dtype: datetime64[ns]
```

In [24]:
```
clean_data['InvoiceDate'].min(), clean_data['InvoiceDate'].max()
```

Out[24]:
```
(Timestamp('2009-12-01 07:45:00'), Timestamp('2010-12-09 20:01:00'))
```

Task 5 — Exploratory Data Analysis

In [25]:
```
## Top Selling Products
top_products = clean_data.groupby('Description')['Quantity'].sum().sort_valu

top_products
```

Out[25]:
```
Description
WHITE HANGING HEART T-LIGHT HOLDER    56915
WORLD WAR 2 GLIDERS ASSTD DESIGNS     54754
BROCADE RING PURSE                    48166
PACK OF 72 RETRO SPOT CAKE CASES      45156
ASSORTED COLOUR BIRD ORNAMENT         44551
60 TEATIME FAIRY CAKE CASES           35806
PACK OF 60 PINK PAISLEY CAKE CASES    31006
JUMBO BAG RED RETROSPOT               29578
SMALL POPCORN HOLDER                  25718
BLACK AND WHITE PAISLEY FLOWER MUG    25685
Name: Quantity, dtype: int64
```

Top Selling Products

This analysis identifies the products with the highest total quantity sold.

Understanding which products sell the most can help reveal popular items and overall purchasing patterns in the dataset.

The results show the top products based on total quantity purchased across all transactions.

In [26]:
```
## Sales by Country
sales_by_country = clean_data.groupby('Country')['Quantity'].sum().sort_valu

sales_by_country.head(10)
```

Out[26]:
```
Country
United Kingdom    4449351
Denmark            229690
Netherlands        183680
EIRE               181428
France             162202
Germany            108740
Sweden              52429
Spain               22856
Switzerland         22255
Australia           20189
Name: Quantity, dtype: int64
```

Sales by Country

Transactions in the dataset come from multiple countries.

By grouping purchases by country, we can observe which regions generate the highest number of sales.

This helps understand the geographic distribution of the retailer's customers.

In [27]:
```
## Monthly Sales Trend
clean_data['YearMonth'] = clean_data['InvoiceDate'].dt.to_period('M')

monthly_sales = clean_data.groupby('YearMonth')['Quantity'].sum()

monthly_sales
```

Out[27]:
```
YearMonth
2009-12    400201
2010-01    370967
2010-02    372771
2010-03    503467
2010-04    352042
2010-05    386297
2010-06    391682
2010-07    325661
2010-08    453590
2010-09    569265
2010-10    598588
2010-11    656324
2010-12    158369
Freq: M, Name: Quantity, dtype: int64
```

Monthly Sales Trend

To examine how sales change over time, transactions were aggregated by month.

This allows us to observe seasonal patterns and changes in purchasing behaviour across the dataset's time period.

In [28]:
```
## Top Customers by Purchase Volume
top_customers = clean_data.groupby('Customer ID')['Quantity'].sum().sort_val

top_customers
```

Out[28]:
```
Customer ID
13902.0    220600
14646.0    170342
13694.0    125893
18102.0    124216
14156.0    108107
14277.0     87830
13687.0     87167
17940.0     75825
14911.0     69722
16754.0     63551
Name: Quantity, dtype: int64
```

Top Customers

Customer level aggregation helps identify which customers contribute the most purchases.

This can be useful for understanding customer behaviour and identifying high-value customers.

Task 6 — Reflection and Planning

Key Insights

1. The majority of transactions come from the United Kingdom, indicating that the retailer's customer base is heavily concentrated in one country.

2. A small number of products account for a large share of total purchases, suggesting that certain items are consistently popular among customers.

3. Customer purchase behaviour varies significantly, with some customers placing very large orders compared to others.

Assumptions and Risks

1. Removing transactions with missing Customer ID assumes that these rows cannot be reliably linked to customers, which may reduce the total amount of data available for analysis.

2. Removing negative quantities and cancelled invoices assumes these represent returns or reversed transactions. If they were incorrectly recorded purchases, removing them could slightly distort the dataset.

Chosen Unit of Analysis

The customer level will be used as the primary unit of analysis. Aggregating transactions by customer allows analysis of purchasing behaviour over time and supports future modelling tasks such as customer segmentation or predicting purchasing patterns.

Next Modelling Direction

In the following weeks, the analysis will focus on modelling customer purchasing behaviour. Possible tasks include identifying high-value customers or predicting future purchase activity using aggregated customer features.

In [ ]:

```

```

In [ ]:

```

```